

09_containers_deployment

Deploying the Local-AI Stack via the containers Submodule in Rootless Podman

Task: T1/main deep-research + design report **Date:** 2026-07-06 **Scope:** LLM serving, VLM, image-gen, video-gen, translation, embeddings, vector DB, memory, Whisper, Tesseract — booted through the containers submodule (digital.vasic.containers) under rootless podman, per §11.4.76 (Containers-Submodule mandate) and §11.4.161 (rootless-container-runtime mandate).
Grounding discipline: §11.4.6 — every host/API claim below is captured ([G- *]) or cited; unproven items are marked UNCONFIRMED:.

0. Grounding facts (captured this session)

ID	Fact
G-HOST-1	Host GPU = RTX 5090, 32 GB VRAM; driver 570.169; CUDA 12.8. nvidia-ctk (NVIDIA Container Toolkit) is ABSENT ; no CDI specs exist (/etc/cdi, /var/run/cdi, ~/.config/cdi empty). Rootless GPU passthrough is therefore not yet possible — this is the #1 blocker.
G-HOST-2	nvcc absent on host (irrelevant for container workloads — the toolchain lives in the -devel image).
G-HOST-3	podman 5.7.1, rootless=true. 64 cores / 251 GiB RAM. No docker.
G-CTN-1	containers submodule at submodules/containers, module digital.vasic.containers, go 1.24+. Packages pkg/boot, pkg/compose, pkg/health, pkg/runtime, pkg/endpoint, pkg/scheduler, pkg/remote read directly.

Blackwell (sm_120) implication (G-HOST-1): the RTX 5090 is compute capability sm_120; every AI image MUST be built on **CUDA ≥ 12.8** with a matching PyTorch cu128 wheel, or kernels fail to load ([vLLM #13306], [vLLM RTX5090 forum]).

1. The containers-submodule API a consumer actually uses (READ, not guessed — §11.4.6)

1.1 Service declaration — pkg/endpoint.ServiceEndpoint

Boot input is `map[string]endpoint.ServiceEndpoint`. Verified fields (`pkg/endpoint/endpoint.go`):

```
type ServiceEndpoint struct {
    Host, Port, URL          string
    Enabled, Required, Remote bool
    HealthPath, HealthType  string           // HealthType:
                          "http"|"tcp"|"grpc"
    Timeout                  time.Duration
    RetryCount               int
    ComposeFile              string           // path to the compose file for this
                          service
    ServiceName              string           // service name within that compose file
    Profile                  string           // compose --profile
    DiscoveryEnabled          bool
    DiscoveryMethod           string           //
                          "dns"|"consul"|"static"
    DiscoveryTimeout          time.Duration
    Discovered               bool
}
```

GAP-1 (load-bearing): ServiceEndpoint has **no GPU/VRAM/device field**. A GPU service is declared exactly like a CPU one — the GPU request lives *inside the compose file the endpoint points at*, not in the Go struct.

1.2 Boot orchestration — pkg/boot

```
func NewBootManager(endpoints
    map[string]endpoint.ServiceEndpoint,
    opts ...BootManagerOption) *BootManager
func (bm *BootManager) BootAll(ctx context.Context)
    (*BootSummary, error)
```

```

func (bm *BootManager) HealthCheckAll(ctx context.Context)
    map[string]error
func (bm *BootManager) Shutdown(ctx context.Context) error

// Functional options (pkg/boot/options.go):
WithRuntime(runtime.ContainerRuntime)
    WithOrchestrator(compose.ComposeOrchestrator)
WithHealthChecker(health.HealthChecker)
    WithDiscoverer(discovery.Discoverer)
WithDistributor(Distributor)
    WithHostManager(remote.HostManager)
WithScheduler(scheduler.Scheduler)          WithProjectDir(dir
    string)
WithLogger(logging.Logger)
    WithMetrics(metrics.MetricsCollector)
WithEventBus(event.EventBus)

```

BootAll runs three phases (pkg/boot/manager.go): **(1) discovery** (skips ! Enabled, resolves DiscoveryEnabled remotes), **(2) compose up** — endpoints are `groupByCompose()` by `ComposeFile` and each group booted via `orchestrator.Up(ctx, compose.ComposeProject{File, Profile})`, **(3) health checks** — `HealthCheckAll` builds `health.HealthTargets` from each endpoint and fails the boot only if a Required service is unhealthy. Returns a `*BootSummary{Started, Remote, Discovered, Failed, Skipped, TotalDuration}`.

1.3 Compose backend — pkg/compose

```

type ComposeOrchestrator interface {
    Up(ctx, project ComposeProject, opts ...UpOption) error
    Down(ctx, project ComposeProject, opts ...DownOption) error
    Status(ctx, project ComposeProject) ([]ServiceStatus, error)
    Logs(ctx, project ComposeProject, service string)
        (io.ReadCloser, error)
}
func NewDefaultOrchestrator(workDir string, logger
    logging.Logger) (*DefaultOrchestrator, error)

```

Backend auto-detection order (`detectComposeCmd`, `pkg/compose/orchestrator.go`): `docker compose` → `docker-compose` → **podman-compose** → `podman compose`. On this host (no docker) it selects `podman-compose` if the `pip` tool is installed, else `podman compose`.

The orchestrator is **podman-aware**: `isPodmanCompose` is set when the command is standalone `podman-compose`; for that backend it omits docker's `--wait` flag and instead **host-side polls `Status()`** until every service is running/healthy

(waitForServices, 120 s default, 1 s tick), and parses **podman-native ps --format json** rather than the docker Go-template (a real bug it fixes). Enable this wait via `compose.WithWait(true)`.

GAP-2 / RISK (cited): when the backend is **podman compose** (the docker-compose provider invoked through the podman CLI), CDI GPU devices declared in a compose `devices: list` are **silently ignored** — the container starts with **no GPU** ([podman #28436], [oneuptime podman-compose GPU]). Standalone **podman-compose (pip)** honors them. The design **MUST** pin the backend to podman-compose.

HelixService (the programmatic compose model, `pkg/compose/helix_project.go`) also has **no GPU field** — only `ResourceLimits{CPUs, Memory, Pids}`. This confirms GAP-1 at the compose layer.

1.4 Health — pkg/health

```
type HealthChecker interface {
    Check(ctx, HealthTarget) *HealthResult
    CheckAll(ctx, []HealthTarget) []*HealthResult
}
func NewDefaultChecker() *DefaultChecker           // pre-loads
    tcp/http/grpc
func (c *DefaultChecker) Register(t HealthType, f
    CheckFunc) // add "custom"
func NewCustomCheckFunc(fn CustomCheckFunc)
    CheckFunc // wrap arbitrary probe

// GPU-specific (pkg/health/gpu.go):
type VRAMProbe interface { Probe(ctx) (freeMB int, err error) }
func NewGPUHealthCheck(p VRAMProbe, minFreeVRAMMB int)
    *GPUHealthCheck
func (c *GPUHealthCheck) Check(ctx)
    error // ErrGPUUnhealthy if free < floor
```

GPUHealthCheck is the **VRAM-budget gate** the topology below leans on: a VRAMProbe (implementable by shelling `nvidia-smi --query-gpu=memory.free --format=csv,noheader,nounits` on the host, or `podman exec ... nvidia-smi`) lets the boot manager refuse to start a resident-tier service unless the card has enough free VRAM.

1.5 Remote/scheduler GPU model (already present — reuse, don't reimplement)

```
pkg/scheduler.GPURequirement{Count, MinVRAMMB, Vendor,
MinCompute, Capabilities} + pkg/remote.GPUDevice{Model,
```

VRAMTotalMB, VRAMFreeMB, ComputeCapability, CUDA/NVENC/NVDEC/Vulkan/OpenCL/ROCM...} + scheduler.ContainerRequirements.GPU *GPURequirement already model GPU-aware **placement across remote hosts**. This is the correct extension point if the stack later spans a second GPU box (CONTAINERS_REMOTE_HOST_N_LABELS=gpu=true). It does **not** help single-host local GPU passthrough (GAP-1/GAP-3).

GAP-3: pkg/runtime.ContainerRuntime (the local podman run path) exposes no --device/CDI option (only pkg/runtime/lxd.go has a generic Devices map). Local GPU containers must go through the **compose file** today, not the programmatic runtime API.

2. Rootless-podman GPU enablement — nvidia-ctk / CDI (fixes G-HOST-1)

This is the mandatory prerequisite; nothing GPU boots until it is done. All steps run **once** on the host.

2.1 Install the NVIDIA Container Toolkit ([NVIDIA install-guide])

nvidia-ctk ships in nvidia-container-toolkit. On an apt host:

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-
  toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/
  deb/nvidia-container-toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/
  nvidia-container-toolkit-keyring.gpg] https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-
  toolkit.list
sudo apt-get update && sudo apt-get install -y nvidia-container-
  toolkit # ≥ 1.18 recommended
```

(RPM/ALT hosts: use the .repo file + dnf install nvidia-container-toolkit, or the distro's native package. The host here is ALT Linux — UNCONFIRMED: whether ALT packages nvidia-container-toolkit; if not, install from the NVIDIA rpm repo or build nvidia-ctk from the libnvidia-container release. Verify with nvidia-ctk --version before proceeding — install exit 0 ≠working binary, per §11.4.102(C).)

2.2 Rootless config: disable cgroup management

Rootless podman cannot manage device cgroups, so the toolkit must be told not to try ([podman-desktop GPU], [NVIDIA install-guide]):

```
sudo nvidia-ctk config --set nvidia-container-cli.no-cgroups --
    in-place
# writes no-cgroups = true into /etc/nvidia-container-runtime/
    config.toml
```

2.3 Generate the CDI spec (the missing artifact — G-HOST-1)

CDI is NVIDIA's **recommended** rootless mechanism ([NVIDIA cdi-support]).

- **Rootless (user-owned spec)** — no root needed at run time ([oneuptime podman GPU]):

```
mkdir -p ~/.config/cdi
nvidia-ctk cdi generate --output=$HOME/.config/cdi/
    nvidia.yaml
```

- **System-wide (root, readable by all users):**

```
sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml
```

- As of toolkit **v1.18.0** a `nvidia-cdi-refresh` systemd service **auto-regenerates** `/var/run/cdi/nvidia.yaml` on toolkit/driver install, upgrade, and reboot ([NVIDIA cdi-support], [NVIDIA release-notes]). Driver *removal* and MIG reconfig still need a manual regenerate.

Podman reads CDI specs from `/etc/cdi` and `/var/run/cdi` by default; for the rootless `~/.config/cdi` path pass `--cdi-spec-dir=$HOME/.config/cdi`.

2.4 Verify (captured-evidence gate — §11.4.5)

```
nvidia-ctk cdi list          # must list nvidia.com/gpu=all and
    nvidia.com/gpu=0
podman run --rm --device nvidia.com/gpu=all --security-
    opt=label=disable \
    nvidia/cuda:12.8.1-base-ubuntu22.04 nvidia-smi
```

`nvidia-smi` inside the container printing the RTX 5090 + driver 570.169 is the runtime signature that closes G-HOST-1 (§11.4.108). `--security-opt=label=disable` (or `--group-add keep-groups`) avoids SELinux/label denial on rootless ([oneuptime podman GPU]).

2.5 Regeneration mechanism (§11.4.77)

The CDI spec is host state that must be re-obtainable: ship scripts/
gpu_cdi_setup.sh running §2.1–2.4 + a .gitignore-meta/nvidia-cdi.yaml
declaring mechanism: nvidia-ctk cdi generate; requires: nvidia-
container-toolkit>=1.18, driver 570.169. **CDI must be regenerated
whenever the driver changes** — wire that into the boot preflight so a stale spec
is caught.

3. Multi-service GPU sharing on ONE 32 GB card (de-risks RZ-03)

The full stack has ~10 GPU-hungry services but one 32 GB card. Naive co-residency
OOMs: time-sliced/CDI-shared processes **share the whole VRAM pool with no
per-process limit** — if two services each grab 20 GB, one OOMs ([spheron GPU
sharing], [vcluster GPU sharing]). Strategy = **budget + tier + on-demand**, not
“start everything.”

3.1 Mechanism choice for a single consumer GPU

- **MIG** — *not available* on GeForce RTX 5090 (MIG is datacenter-only). Rule it out.
- **Time-slicing (default under CDI)** — multiple containers see the full GPU; concurrency by context-switch, **no memory isolation** ([nvidia gpu-sharing], [scaleops]). Acceptable only if the sum of resident VRAM budgets < 32 GB.
- **MPS (Multi-Process Service)** — concurrent kernel execution **with per-client memory caps** (CUDA_MPS_PINNED_DEVICE_MEM_LIMIT), which **reduces OOM** ([medium MPS], [nvidia consolidating]). Best for the always-resident small services (embeddings + Whisper) that must answer concurrently.
- **On-demand load/unload** — the real lever for the big models:
 - **vLLM Sleep Mode** (v0.11+): level-1 offloads weights to CPU RAM, level-2 discards them; wake is **18–200× faster than a cold reload** ([vLLM sleep-mode]). Ideal for the LLM/VLM tier — keep the server process warm, evict weights between jobs.
 - **Ollama** OLLAMA_KEEP_ALIVE + keep_alive:0 unloads a model to free VRAM; it will juggle models in/out of a too-small card at the cost of swap latency ([runaihome ollama], [sumguy ollama]).
 - **llama.cpp router / -n-gpu-layers** for dynamic model switching without a restart ([glukhov llama-server]).

3.2 Co-resident vs on-demand policy for THIS card (VRAM budget, 32 GB)

Tier	Services	Policy	Rough VRAM
Always-resident (small, latency-critical, concurrent)	embeddings (e.g. bge/nomic), Whisper (base/small)	resident, optionally under MPS with per-client caps	~2–4 GB total
Warm-swappable (one big model at a time)	LLM serving (vLLM/Ollama), VLM	one active , others in vLLM sleep /Ollama keep-alive; a small router picks	12–24 GB for the active model
On-demand burst (heavy, infrequent, exclusive)	image-gen (SDXL/Flux), video-gen (SVD/Wan)	started only for a job , single-owner of remaining VRAM, torn down after (§11.4.119)	10–24 GB while running
CPU-only (no GPU)	vector DB (Qdrant/pgvector), memory store, translation (if CPU MT), Tesseract OCR	never touch the GPU	0

Rules: (1) sum of **resident** budgets < ~6 GB, leaving ≥ 26 GB for one warm/burst model; (2) **never** two burst services (image-gen + video-gen) resident simultaneously — serialize them as single-GPU-owner (§11.4.119); (3) every GPU service sets an explicit cap (--gpu-memory-utilization in vLLM, OLLAMA_GPU_OVERHEAD, CUDA_MPS_PINNED_DEVICE_MEM_LIMIT) so time-sliced neighbours can't oversubscribe; (4) the GPUHealthCheck VRAM floor (§1.4) gates each resident/warm start.

4. Design: containers-submodule-driven topology for the full stack

4.1 Layout (compose profiles = tiers)

One compose file `deploy/ai-stack.compose.yaml` with **profiles** so the boot manager brings up only the wanted tier; `ServiceEndpoint.Profile` selects it, and `groupByCompose` keeps one file → one group.

cpu-core	profile → qdrant, pgvector/postgres, memory, translation, tes
gpu-resident	profile → embeddings, whisper
gpu-warm	profile → llm (vLLM sleep) , vlm
gpu-burst	profile → image-gen, video-gen

4.2 A GPU service entry (compose) — the CDI device shape

```
services:
  vllm:
    image: <registry>/helix-vllm-cu128:<sha>      # CUDA 12.8,
    torch cu128, sm_120
    profiles: ["gpu-warm"]
    devices:
      - nvidia.com/gpu=all                        # CDI ref from §2.3
    security_opt: [ "label=disable" ]
    environment:
      - VLLM_FLASH_ATTN_VERSION=2                # FA3 not yet on Blackwell
        ([vLLM RTX5090 forum])
      - CUDA_VISIBLE_DEVICES=0
    command: ["--gpu-memory-utilization","0.85","--enable-sleep-
    mode"]
    volumes:
      - ai-weights:/models                        # large-weight volume, §4.4
    healthcheck:                                  # host-side wait (podman-
      compose path) polls this
      test: ["CMD-SHELL","curl -sf http://localhost:8000/health"]
      interval: 10s
```

Because `devices: nvidia.com/gpu=all` is silently dropped by `podman compose (GAP-2)`, the **boot backend MUST be podman-compose (pip)**. Enforce it: install `podman-compose`, and `preflight-assert detectComposeCmd()` resolved to it (not `podman compose`).

Alternatively use the `deploy.resources.reservations.devices` form — but the `devices:+CDI` form is the reliable rootless path ([oneuptime podman-compose GPU]).

4.3 Boot wiring (consumer Go, ~15 lines)

```
orch, _ := compose.NewDefaultOrchestrator(projectDir,
    log)           // resolves podman-compose
hc       := health.NewDefaultChecker()
hc.Register(health.HealthCustom,
    health.NewCustomCheckFunc(gpuVRAMGate)) // wraps
    GPUHealthCheck
bm := boot.NewBootManager(endpoints,
    boot.WithOrchestrator(orch), boot.WithHealthChecker(hc),
    boot.WithProjectDir(projectDir), boot.WithLogger(log))
summary, err :=
    bm.BootAll(ctx) // discovery → podman-compose up (per
    profile) → health
```

endpoints is a map[string]ServiceEndpoint where every GPU service shares ComposeFile: "deploy/ai-stack.compose.yaml" and differs by Profile + ServiceName + health Port. WithWait-style readiness is automatic on the podman-compose path.

4.4 Large-weight volume strategy (§11.4.77 re-obtain)

Model weights (tens–hundreds of GB) are **never** git-versioned. Use a named podman volume ai-weights (or a host bind under a fast NVMe) mounted read-write into each GPU service. Re-obtain mechanism per weight family, declared in .gitignore-meta/ai-weights.yaml: HuggingFace Hub pull (huggingface-cli download <repo> --local-dir /models/<name>), Ollama registry pull (ollama pull <model>), or the dedicated dependencies/HuggingFace_Hub/dependencies/Ollama submodules already in the tree. Ship scripts/fetch_ai_weights.sh invoked from setup.sh; a preflight gate checks the weight dir is populated or emits a .regenerated/<slug>.ok stamp. Integrity via sha256 where the source publishes it.

4.5 Extensions to PR upstream into containers (§11.4.74 — extend, don't reimplement)

The submodule is **90% sufficient**; three additive primitives close the gaps (each project-agnostic, decoupled per §11.4.28): 1. **compose.HelixService.Devices []string** (+ emit devices: and security_opt: label=disable in generated compose) — closes GAP-1 for the programmatic compose model. CDI-string valued, vendor-neutral. 2. **runtime GPU/device option** — a WithDevices([]string) / RunOptions.Devices on the local ContainerRuntime.Run path so podman run --device nvidia.com/gpu=all is expressible without a compose file (closes GAP-3). 3. **compose.NewDefaultOrchestrator backend pin / preflight** — an option to

require podman-compose (reject podman compose) when any service requests a CDI device, turning GAP-2 from a silent no-GPU into a hard, captured failure. A **host-side VRAMProbe impl** (shell nvidia-smi) is also worth upstreaming so GPUHealthCheck works out-of-the-box.

Each extension ships with a Challenge that boots a real CUDA-12.8 container and asserts nvidia-smi sees the 5090 (anti-bluff, §11.4.76(5)).

5. Top risks

1. **R1 — GPU passthrough entirely blocked until §2 is done (G-HOST-1).** No AI service can start; nvidia-ctk absent + zero CDI specs. Mitigation: §2 is the first, gating task; verify with the §2.4 `podman run ... nvidia-smi` runtime signature before any stack boot. ALT-Linux packaging of nvidia-container-toolkit is UNCONFIRMED: and may need the NVIDIA rpm repo or a from-source nvidia-ctk.
2. **R2 — silent no-GPU via wrong compose backend (GAP-2, cited).** podman compose drops CDI devices:; the submodule's auto-detect could pick it. Mitigation: install + pin podman-compose (pip); preflight-assert the resolved backend; upstream extension #3.
3. **R3 — single-card VRAM oversubscription / OOM (RZ-03).** 32 GB shared, no MIG, time-slicing has no memory isolation. Mitigation: the tiered resident/warm/burst budget (§3.2), explicit per-service VRAM caps, vLLM sleep + Ollama keep-alive for warm-swap, single-GPU-owner serialization of image-gen vs video-gen (§11.4.119), and the GPUHealthCheck VRAM floor gating each start.

Runner-up: Blackwell/sm_120 image breakage — every AI image must be CUDA 12.8 + torch cu128 + VLLM_FLASH_ATTN_VERSION=2, else kernels fail ([vLLM #13306]).

Sources verified 2026-07-06

Code (read this session): submodules/containers/pkg/{boot/manager.go,boot/options.go,boot/result.go,compose/orchestrator.go,compose/options.go,compose/types.go,compose/helix_project.go,health/gpu.go,health/checker.go,health/custom.go,health/types.go,endpoint/endpoint.go,scheduler/gpu.go,scheduler/types.go,remote/gpu.go} + submodules/containers/CLAUDE.md, .env.example.

Web: - NVIDIA Container Toolkit — Installing: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html> - NVIDIA Container Toolkit

— CDI support: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/cdi-support.html> - NVIDIA Container Toolkit — Release notes (v1.18 nvidia-cdi-refresh): <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/release-notes.html> - Podman Desktop — GPU container access (rootless, no-cgroups): <https://podman-desktop.io/docs/podman/gpu> - OneUptime — Run NVIDIA GPU containers with Podman (rootless CDI, ~/.config/cdi, label=disable): <https://oneuptime.com/blog/post/2026-03-18-run-nvidia-gpu-containers-podman/view> - OneUptime — GPU access in podman-compose (devices: nvidia.com/gpu=all): <https://oneuptime.com/blog/post/2026-03-17-use-gpu-access-podman-compose/view> - podman #28436 — CDI GPU devices not passed through with podman compose: <https://github.com/podman-container-tools/podman/issues/28436> - podman #19338 — compose driver: cdi not supported: <https://github.com/containers/podman/issues/19338> - Spheron — Run multiple LLMs on one GPU (MIG/time-slicing/MPS): <https://www.spheron.network/blog/run-multiple-llms-one-gpu-mig-time-slicing-guide/> - vcluster — DIY GPU sharing (time-slicing has no memory isolation): <https://www.vcluster.com/blog/diy-gpu-sharing-in-kubernetes> - Medium/Zanotti — Increase GPU utilization with NVIDIA MPS (per-client mem cap): <https://medium.com/data-science/how-to-increase-gpu-utilization-in-kubernetes-with-nvidia-mps-e680d20c3181> - NVIDIA — Time-slicing GPUs (GPU Operator): <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-sharing.html> - vLLM Sleep Mode (18–200× faster wake): <https://blog.vllm.ai/2025/10/26/sleep-mode.html> - Ollama VRAM unloading / keep-alive (2026): <https://www.runaihome.com/blog/ollama-model-keeps-reloading-vram-fix-2026/> - llama-server router mode (dynamic model switch): <https://www.glukhov.org/llm-hosting/llama-cpp/llama-server-router-mode/> - vLLM #13306 — RTX 5090 / CUDA 12.8 support: <https://github.com/vllm-project/vllm/issues/13306> - vLLM RTX5090 working setup (torch 2.9 cu128, FA v2): <https://discuss.vllm.ai/t/vllm-on-rtx5090-working-gpu-setup-with-torch-2-9-0-cu128/1492>

Negative findings (§11.4.99): NVIDIA's official install/CDI pages do NOT document the rootless ~/.config/cdi path or no-cgroups=true — those come from Podman Desktop docs + the OneUptime guide (secondary authoritative sources). ALT-Linux nvidia-container-toolkit packaging is unverified (UNCONFIRMED:).